UNIVERSITY OF INFORMATION TECHNOLOGY AND SCIENCES

Course Code: CSE0611324

Course Title: Microprocessors, Microcontrollers and Assembly Language Lab

---

Project Proposal

Submitted to:

Faculty Name	Designation
Tahmina Yeasmin Lima	Lecturer
Abrar Mahbub Tanim	Lecturer

Submitted by:

Name	ID	Other Details
Jannatul Ferdus	0432410005101058	Department of CSE Autumn 2026
Gaus Saraf Murady	0432410005101088	
Sadia Islam	0432410005101096	

University of Information Technology and Science

October 10, 2026

UITS
UNIVERSITY OF INFORMATION TECHNOLOGY AND SCIENCES

“Microprocessor-Controlled Embedded NAS & Static Web Hosting Node”

1. Domain:

Embedded Systems, Internet of Things (IoT), and Networked Microprocessors.

2. Project Overview & Objectives

2.1 Problem Statement

Traditional Network Attached Storage (NAS) units and web hosting servers rely on x86 or high-performance ARM application processors, which consume substantial power (15W–60W) and require complex operating systems.

2.2 Proposed Solution

This project designs a low-power, solid-state micro-NAS and web server node built around the **ESP32** microcontroller. Utilizing FreeRTOS, a dedicated hardware SPI/SDMMC interface, and asynchronous TCP handling, the system functions as a localized wireless file storage server that can easily expand into hosting static web pages, client-side Single-Page Applications (SPAs), and sensor dashboards.

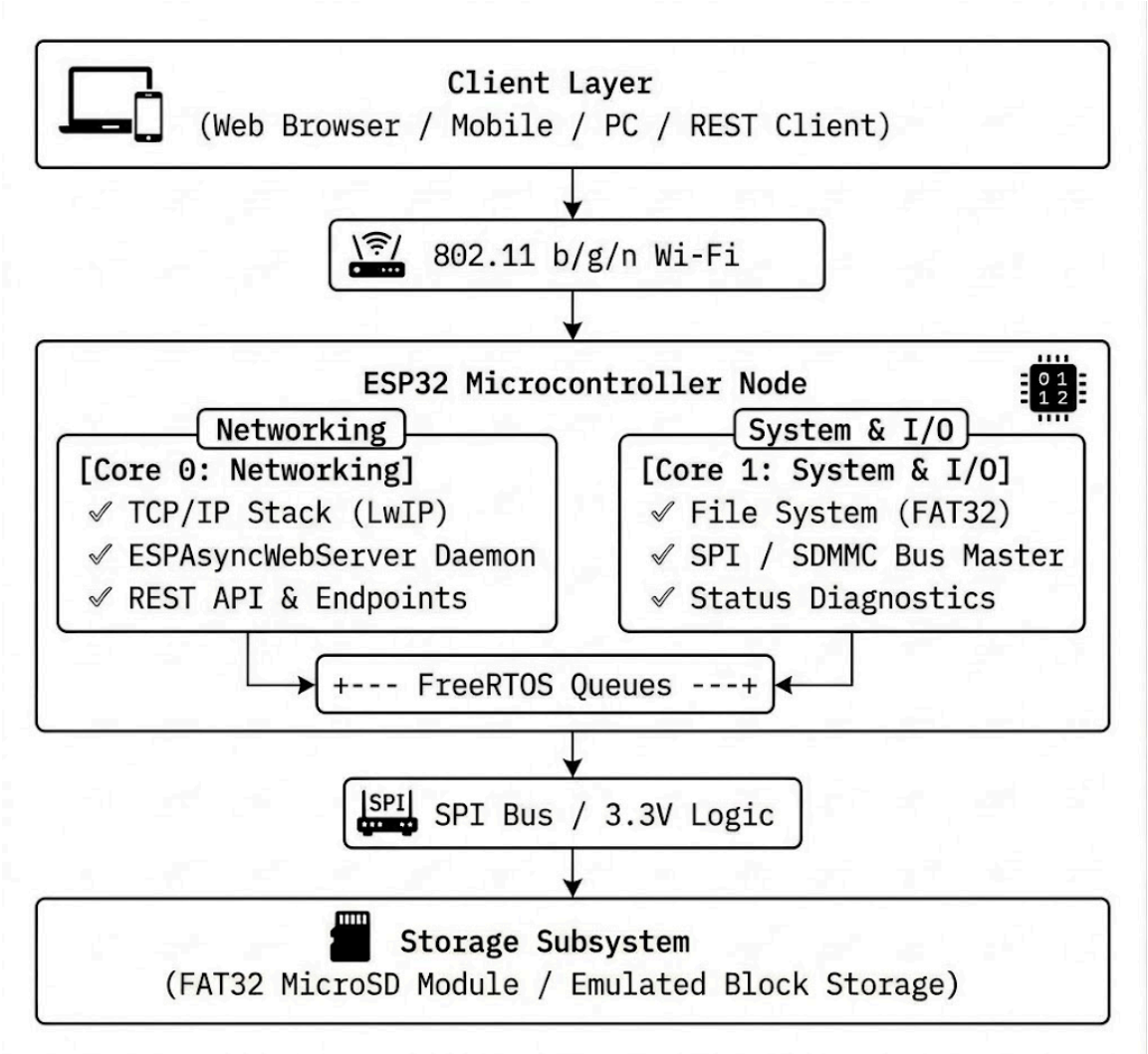
2.3 Feasibility Analysis

- **Architectural & Computational Feasibility:** Dual-core Xtensa architecture permits clean task separation (Core 0 manages the Wi-Fi/TCP stack while Core 1 handles SPI storage operations via FreeRTOS), preventing network timeouts during disk read/write loops.
- **Simulation Feasibility:** Full-system emulation is entirely viable using Wokwi, which executes compiled target machine code and tunnels network sockets locally via a Private Gateway, allowing complete HTTP server and file-streaming validation without physical hardware.

2.4 Core Objectives

- Implement a wireless, localized file storage architecture on an embedded platform.
- Design an intuitive HTTP dashboard for file manipulation (viewing directory structure, downloading files, reporting free/used storage).
- Enable dual-purpose functionality: serving file streams alongside static web assets (HTML5/CSS/JavaScript).
- Evaluate system throughput, memory footprints, and architectural constraints on resource-constrained embedded systems.

3. System Architecture & Working Principle



3.1 Hardware Requirements

- **Microcontroller:** ESP32 NodeMCU (Xtensa dual-core 32-bit LX6 microprocessor, up to 240 MHz, 520 KB SRAM).
- **Storage Interface:** MicroSD Card Module over Serial Peripheral Interface (SPI).
- **Storage Media:** 8 GB to 32 GB MicroSD Card formatted with standard FAT32.
- **Network Controller:** On-chip 2.4 GHz 802.11 b/g/n Wi-Fi transceiver.

3.2 Functional Modes

1. **NAS / File Management Mode:** Clients query endpoints to inspect directory contents and stream binary chunks from storage to local disks via HTTP octet-streams.
2. **Static Web Hosting Mode:** Assets placed in a dedicated root directory (e.g., /www/) are served to web browsers. The node acts as a zero-cloud, off-grid documentation or local dashboard host.

3.3 Software and Technologies

- **Microcontroller Firmware:** Arduino Core / ESP-IDF Framework.
- **Real-Time Operating System:** FreeRTOS (Dual-core task partitioning between Wi-Fi stack and I/O operations).
- **Network Protocol Stack:** lwIP TCP/IP stack, ESPAsyncWebServer daemon for asynchronous socket and HTTP routing.
- **File System:** FAT32/FATFS block device driver for MicroSD storage management.
- **Prototyping & Simulation Environment:** Wokwi Browser-Based Instruction-Accurate Simulator (utilizing WebAssembly execution and Private Gateway tunneling).

4. Implementation Procedures

Step 1: Circuit Setup & Interface Pinout

Connect the MicroSD card module to the ESP32's hardware VSPI bus:

ESP32 Pin	SD Module Pin	Function	Description
3V3	VCC	Power	Logic/Module Supply (3.3V)
GND	GND	Ground	Common Ground
GPIO 5	CS	Chip Select (SS)	SPI Slave Select
GPIO 23	MOSI / DI	Data Input	Master Out, Slave In
GPIO 19	MISO / DO	Data Output	Master In, Slave Out
GPIO 18	SCK / CLK	Clock	SPI Synchronous Clock

Step 2: Firmware Implementation

The firmware runs an asynchronous web server that multiplexes HTTP request dispatching and block device streaming.

```
C++
#include <WiFi.h>
#include <ESPAsyncWebServer.h>
#include <SPI.h>
#include <SD.h>

#define SD_CS_PIN 5

const char* ssid = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";
AsyncWebServer server(80);

String formatBytes(size_t bytes) {
  if (bytes < 1024) return String(bytes) + " B";
  if (bytes < (1024 * 1024)) return String(bytes / 1024.0, 1) + " KB";
  return String(bytes / (1024.0 * 1024.0), 1) + " MB";
}

void setup() {
  Serial.begin(115200);

  // Initialize Storage Peripheral
  if (!SD.begin(SD_CS_PIN)) {
    Serial.println("[ERROR] Storage device initialization failed.");
    return;
```

```
  }
  Serial.println("[STORAGE] FAT32 Storage Mounted.");

  // Connect to Wi-Fi Subnet
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(200);
    Serial.print(".");
  }
  Serial.printf("\n[NET] Server Active at: http://%s/\n",
WiFi.localIP().toString().c_str());

  // Web Hosting Endpoint: Serves static files directly from /www/ on SD
  server.serveStatic("/site", SD, "/www/").setDefaultFile("index.html");

  // NAS File Explorer Endpoint
  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request) {
    String html = "<html><head><title>ESP32 Storage Node</title></head><body>";
    html += "<h2>Embedded Storage Node</h2>";
    html += "<p>Total: " + formatBytes(SD.totalBytes()) + " | Used: " +
formatBytes(SD.usedBytes()) + "</p>";
    html += "<ul>";

    File root = SD.open("/");
    File file = root.openNextFile();
    while (file) {
      if (!file.isDirectory()) {
        String name = String(file.name());
        html += "<li><a href='/get?file=" + name + "'>" + name + "</a> (" +
formatBytes(file.size()) + ")</li>";
      }
      file = root.openNextFile();
    }
    html += "</ul></body></html>";
    request->send(200, "text/html", html);
  });

  // NAS File Stream / Download Endpoint
  server.on("/get", HTTP_GET, [](AsyncWebServerRequest *request) {
    if (request->hasParam("file")) {
      String path = "/" + request->getParam("file")->value();
      if (SD.exists(path)) {
        request->send(SD, path, "application/octet-stream", true);
        return;
      }
    }
    request->send(404, "text/plain", "File Not Found");
  });

  server.begin();
}

void loop() {
  // Asynchronous handling runs in background tasks on Core 0
}
```

Step 3: Deployment & Testing Protocol

1. **Simulation Stage (Wokwi):**
   - Construct the diagram with ESP32 and MicroSD module components.
   - Verify initialization via the virtual serial console.
   - Test simulated HTTP requests across the internal bridged network adapter.
2. **Physical Hardware Assembly:**
   - Breadboard the module using short, direct jumpers to minimize SPI line capacitance.
   - Format the storage card to FAT32 (32 KB cluster size allocation).
   - Verify read/write integrity by uploading and downloading binary files of various sizes (10 KB to 5 MB).

5. Project Scope and Limitations:

5.1 Electrical & Power Stability

- **Inrush Current Surges:** MicroSD cards exhibit sudden current spikes of **up to 150–200 mA** during flash page write cycles. When combined with Wi-Fi transmission bursts (up to 250 mA), total peak draw can exceed 500 mA.
- **Brownout Resets:** Powering the setup solely through an unpowered USB hub or long breadboard jumpers frequently triggers the ESP32's on-chip brownout detector, causing continuous reboot cycles.
- **Mitigation:**
  - Place a **10 µF electrolytic capacitor** in parallel with a **0.1 µF ceramic decoupling capacitor** directly across the SD module's VCC and GND terminals.
  - Use a dedicated external 5V/2A DC power supply connected directly to the ESP32's VIN pin.

5.2 Microprocessor Memory Constraints & Latency

- **SRAM Budget:** While the ESP32 features 520 KB of internal SRAM, the usable heap after loading the Wi-Fi stack and TCP buffers drops to roughly **250–300 KB**.
- **Stream Buffering:** Files must never be buffered entirely into RAM before transmission. The firmware must use chunked streaming (typically 1 KB to 4 KB transfer blocks).
- **Concurrency Limits:** Every open TCP connection allocates an internal PCB (Protocol Control Block) and socket buffers. The system can handle 4 to 8 concurrent clients; heavy request bursts will exhaust the heap and drop packets.

- **Throughput Limits:** Standard 1-bit SPI mode yields sustained transfer rates of roughly **200 KB/s to 450 KB/s**. While fully adequate for static web hosting and configuration transfers, it cannot replace Gigabit or multi-megabyte consumer NAS solutions.

5.3 Simulation vs. Physical Implementation Limitations

Operational Parameter	Simulation Environment (Wokwi)	Physical Hardware Implementation
Storage Medium	Emulated block device backed by browser memory.	Physical flash card with sector wear, write latency, and sector re-mapping.
USB Host / External HDD	Cannot emulate external USB hard drives or USB Mass Storage Class (MSC) bridges.	Classic ESP32 lacks USB Host OTG hardware; interfacing a real HDD requires an ESP32-S3 or a hardware controller IC (e.g., CH376S).
Signal Integrity & Noise	Perfect digital logic transitions with zero line capacitance or cross-talk.	High-frequency SPI lines (10–20 MHz) suffer from signal degradation and ground bounce if breadboard leads are too long.
Network Latency & NAT	Bridged through a simulated private subnet (Wokwi-GUEST) with artificially low jitter.	Subject to real-world RF interference, 2.4 GHz channel congestion, dynamic DHCP delays, and router packet drops.
Thermal & Power Behaviors	Zero thermal throttling; simulated power rail is infinitely stable.	ESP32 core temperature rises under sustained Wi-Fi operations; vulnerable to supply brownout faults.

6. Expected Outcome & Evaluation Criteria

1. **Simulation Demonstration:** Functional Wokwi project link demonstrating storage initialization, virtual web server availability, and log verification on the serial monitor.
2. **Physical Prototype:** Operating ESP32 circuit wired to an external SD module serving an embedded web portal accessible by mobile devices over local Wi-Fi.
3. **Benchmarking Report:** Empirical measurements detailing:
   - Read and write throughput (KB/s) across varying file payloads.
   - Free heap memory transitions under multiple concurrent HTTP connections.
   - Average latency and response times when serving static web assets.